

AI FARM ROBOTICS FACTORY

A LIVING CODE

STEAM_x LAB

STM32 SD CARD

Prepare By: STEAM_x LAB

12.12.2023

Content

1. Introduction	1
1.1. Hardware Required	1
2. STM32 SD Card Interfacing.....	1
2.1. Connection Diagram.....	1
2.2. SD Card Connection (STM32 SPI)	1
2.3. UART Connection	2
3. Project Creation	3
3.1. RCC	3
3.2. SYS	4
3.3. SPI (Serial Peripheral Interface).....	5
3.4. USART (Universal Synchronous Asynchronous Receiver Transmitter)	6
3.5. FATFS	7
3.6. Clock Configuration	8
4. Source Code	8
4.1. Fatfs_sd.h	8
4.2. Fatfts_sd.c	9
4.3. STM32f1xx_it.c.....	16
4.4. User_diskio.c	17
4.5. Example code	18
5. Result Testing	19
6. Resources	20

Figure 2.1: Diagram SD CARD	2
Figure 2.2: Diagram USB UART.....	3
Figure 3.1: STM32CubeMx Configuration RCC.....	4
Figure 3.2: STM32CubeMx Configuration SYS.....	4
Figure 3.3: STM32CubeMx Configuration SPI	5
Figure 3.4: STM32CubeMx Configuration PIN CS.....	6
Figure 3.5: STM32CubeMx Configuration USART	6
Figure 3.6: STM32CubeMx Configuration FATFS	7
Figure 3.7: STM32CubeMx Clock Configuration	8
Figure 4.1: Library Fatfs_sd.h	8
Figure 4.2: Library Fatfs_sd.c.....	15
Figure 4.3: STM32f1xx_it.c.....	16
Figure 4.4: user_diskio.c	17
Figure 4.5: Code main.c	18
Figure 5.1: USB-Serial UART	19
Figure 5.2: USB-Serial UART PUTTY.....	19
Figure 5.3: USB Drive (F) from SD CARD.....	20

1. Introduction

This is the Series of tutorials on the STM32 Microcontroller. The aim of this series is to provide easy and practical examples that anyone can understand. In this article, we are going to interface the SD CARD with STM32 (STM32 SD CARD) using SPI communication. In this demo, we have used the STM32F401cc Black Pill board. You can also try this method with other STM32 boards.

1.1. Hardware Required

- [STM32F103C8T6 Blue Pill Board](#)
- [SD Card Reader](#)
- [USB to Serial Converter \(For debug prints\)](#)

2. STM32 SD Card Interfacing

We have assumed that you have installed the STM32CubeIDE and know how to use that IDE. If you don't know also fine you can follow the below procedure.

2.1. Connection Diagram

The SD Card is connected to the STM32 through SPI. If you want to learn about SPI, you can check [this tutorial](#). The SD Card connection is given below.

Note: Make sure you are giving 5V to the VCC of the SD Card reader. If you lesser than that, it won't work.

2.2. SD Card Connection (STM32 SPI)

SD Card Reaser	STM32F103C8
VCC	5V
GND	GND
CS	PA4
SCK	PA5
MISO	PA6
MOSI	PA7

- Diagram USB UART

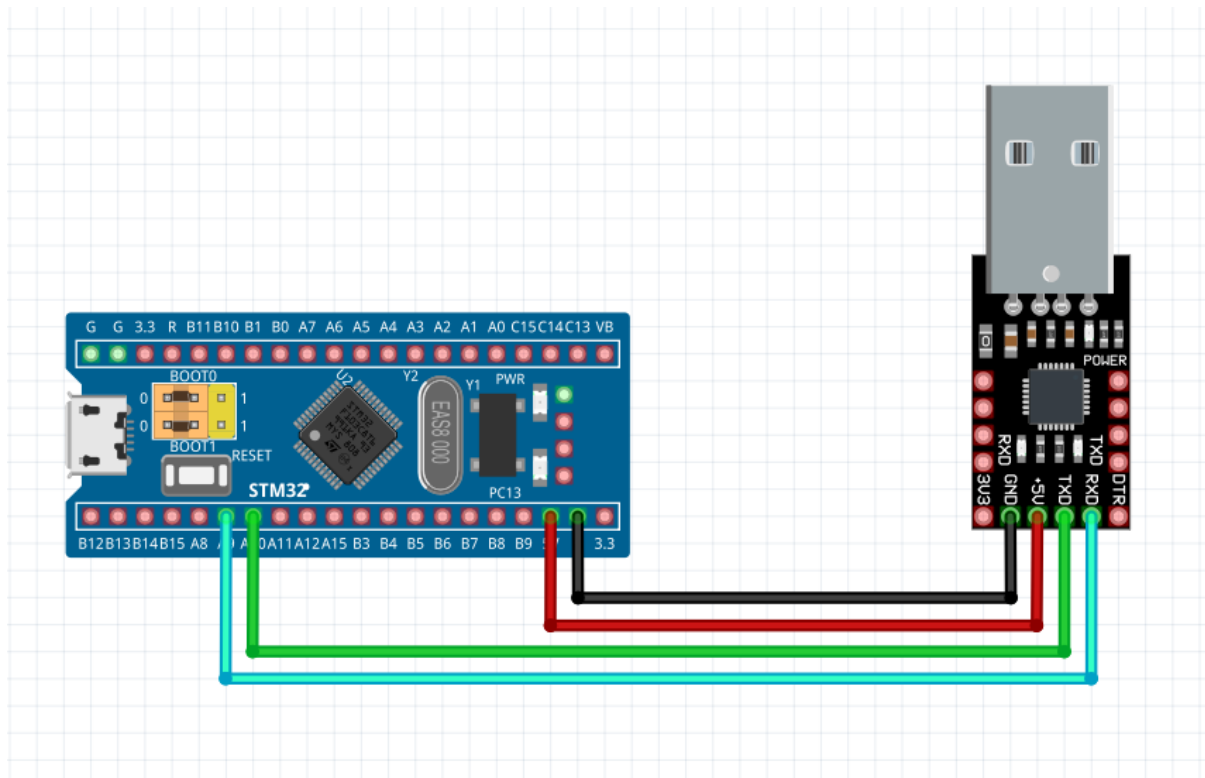


Figure 2.2: Diagram USB UART

3. Project Creation

3.1. RCC

Create the new project in STM32CubeIDE. In the .ioc file pinout and configuration, click **System Core** ⇒ **RCC**. Select the **Mode** as **Crystal/Ceramic Resonator**



Create the new project in STM32CubeIDE. In the .ioc file pinout and configuration, click **System Core** $\Rightarrow$ **SYS**. Select the **Mode** as **Serial Wire**



3.3. SPI (Serial Peripheral Interface)

Create the new project in STM32CubeIDE. In the .ioc file pinout and click Connectivity

⇒ **SPI1**. Select the **Mode** as **Full-Duplex Master**. Then click **PA4** in the Pinout view and set that as a **GPIO_Output**. This GPIO will be used for **Chip Select (CS)**.

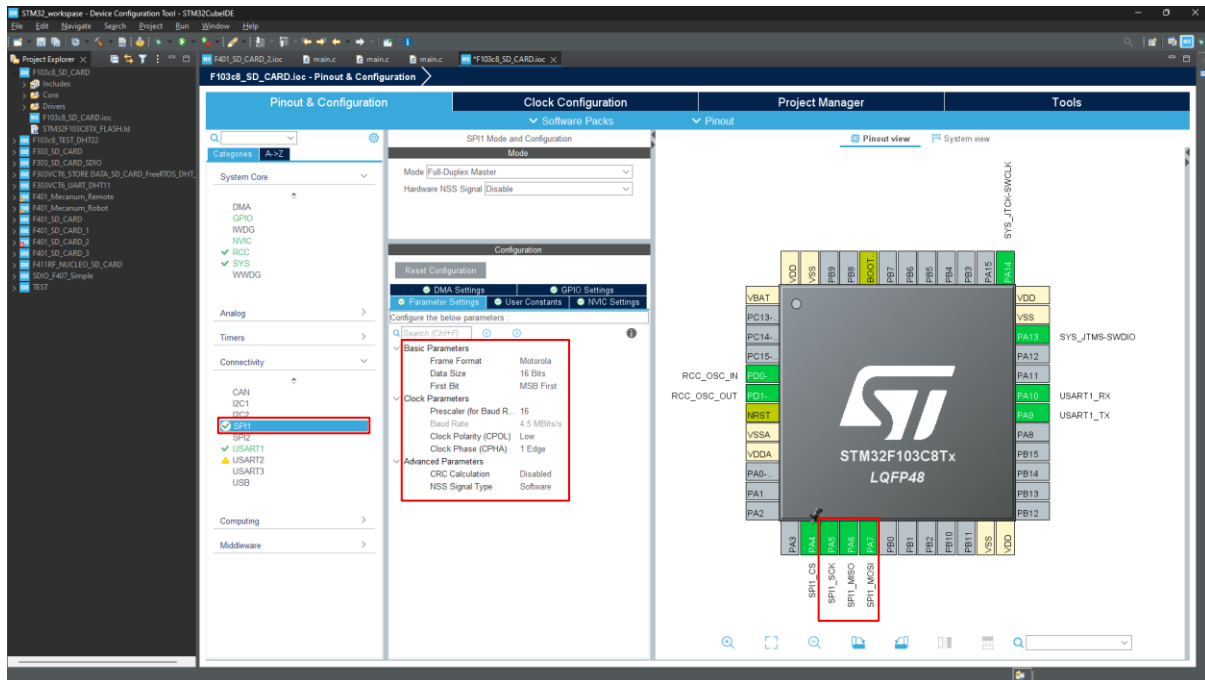


Figure 3.3: STM32CubeMx Configuration SPI

- Chip Select (CS) GPIO_Outout.

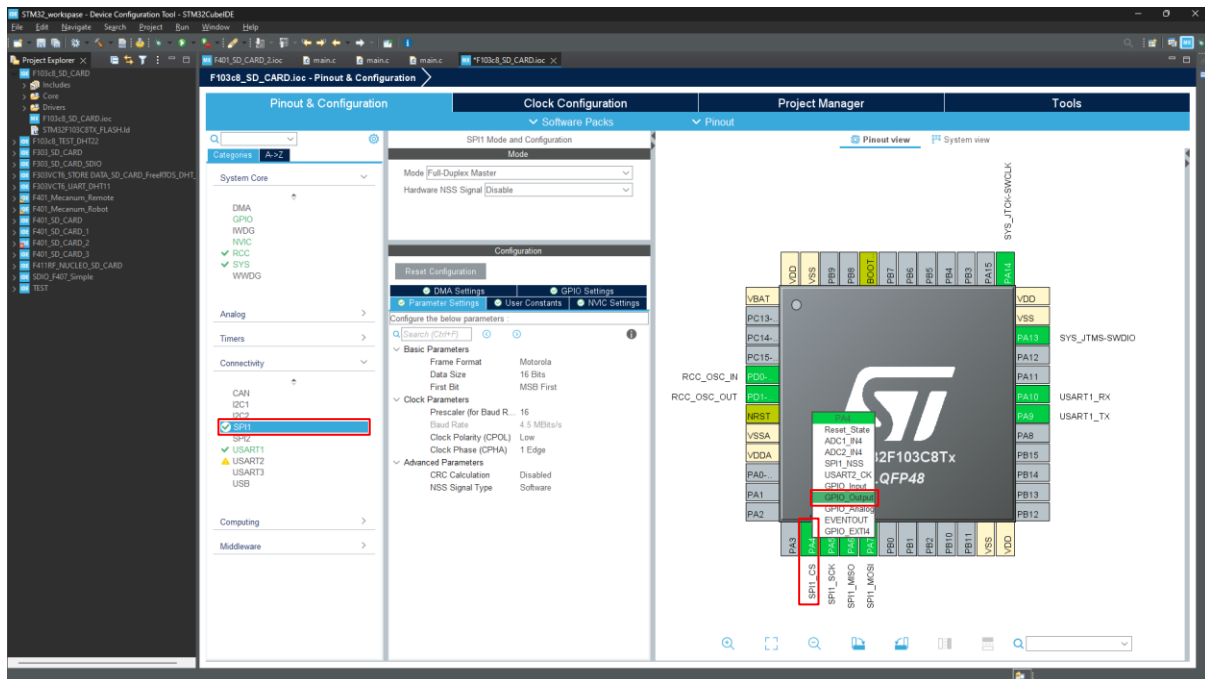


Figure 3.4: STM32CubeMx Configuration PIN CS

3.4. USART (Universal Synchronous Asynchronous Receiver Transmitter)

Create the new project in STM32CubeIDE. In the .ioc file pinout and click Connectivity

⇒ **USART1**. Select the **Mode** as **Asynchronous**. The click **Parameter Settings** and Select **Basic Parameters** Baud Rate **115200 Bits/s**.

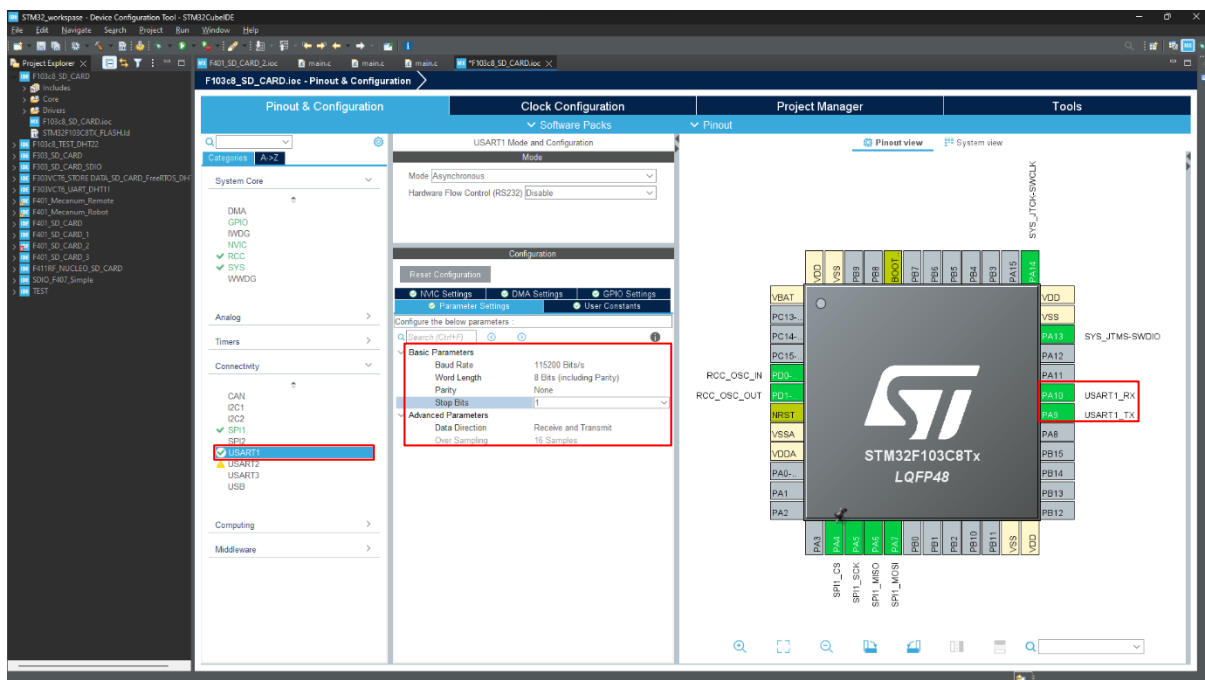


Figure 3.5: STM32CubeMx Configuration USART

3.5. FATFS

FATFS is a lightweight software library for microcontrollers and embedded systems that implements FAT/exFAT file system support.

- Now we will enable the FATFS. Click Middleware → FATFS. Then tick User-defined. In the FATFS Configuration, set USE_LEN as Enable with static working buffer on the BSS to use the long file name. Then set the MAX_SS as 4096

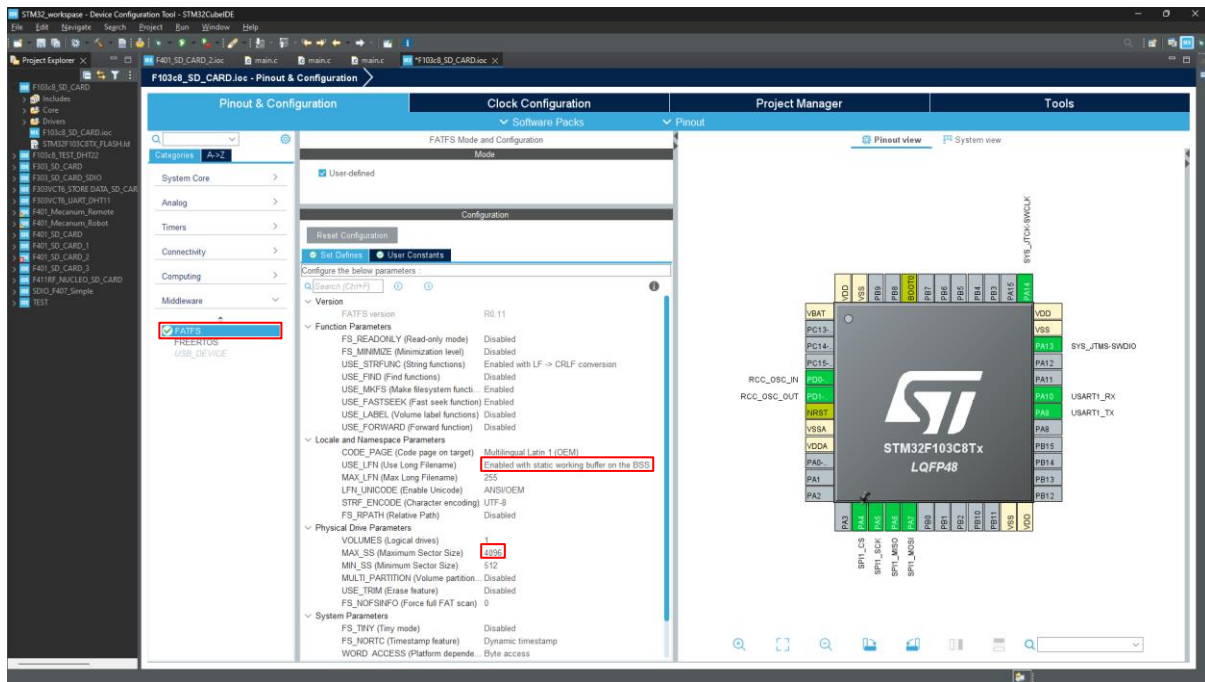


Figure 3.6: STM32CubeMx Configuration FATFS

3.6. Clock Configuration

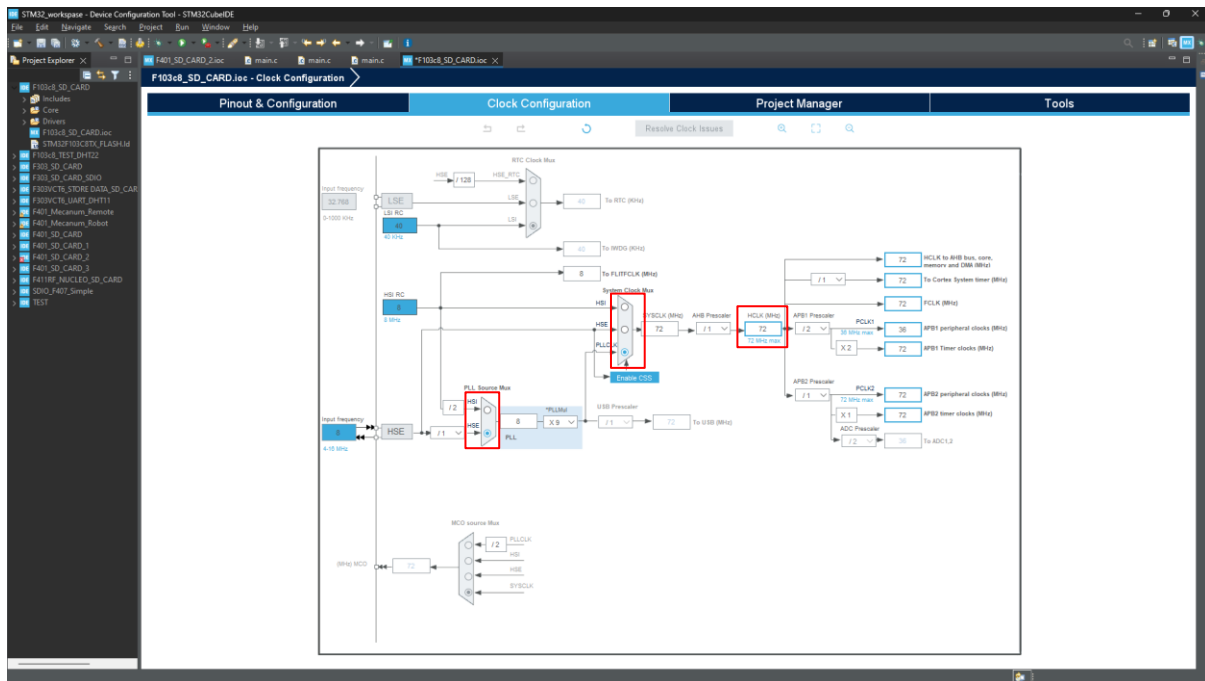


Figure 3.7: STM32CubeMx Clock Configuration

4. Source Code

Now we have the auto-generated code. We will add our code on top of this auto-generated code. You can get the complete code of this example project on [Google-drive](#).

4.1. Fatfs_sd.h

- Create a new header file name [fatfs_sd.h](#) under the Core/Inc. the copy and paste the below code to that file here [fatfs_sd.h](#).

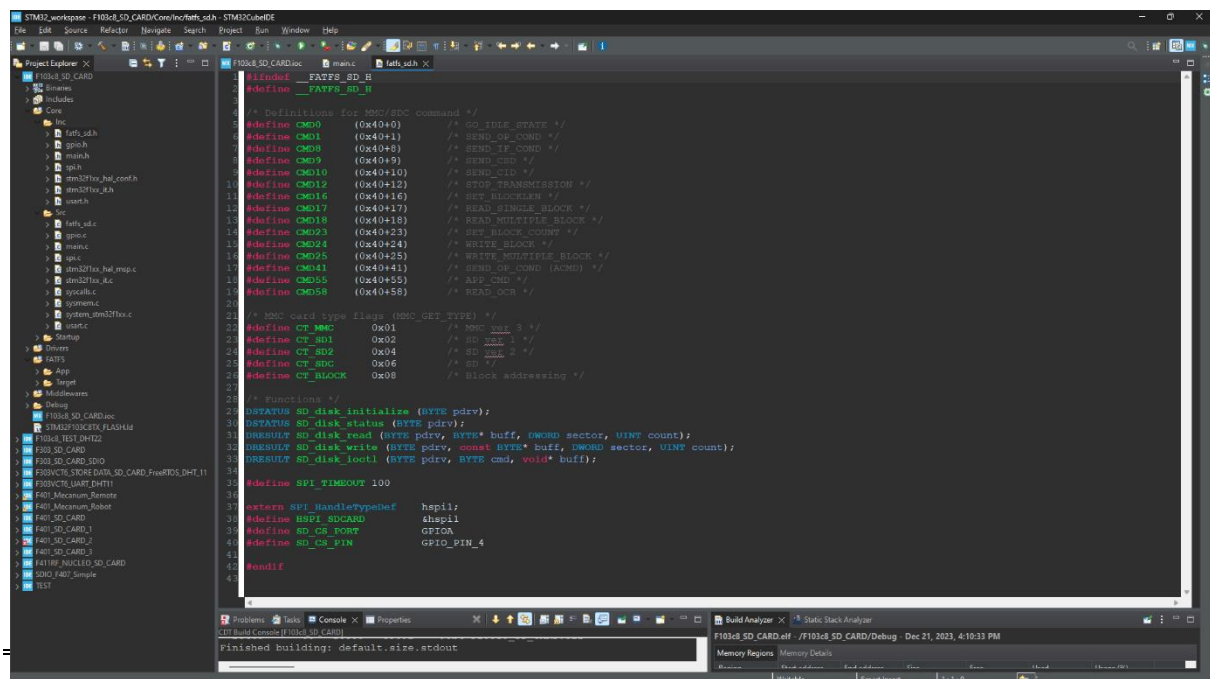


Figure 4.1: Library Fatfs_sd.h

4.2. Fatfts_sd.c

- Create a new header file name [fatfts_sd.c](#) under the Core/Src. Then copy and paste the below code to that file here [fatfts_sd.c](#)

```

1  #define TRUE 1
2  #define FALSE 0
3  #define bool BYTE
4
5  #include "stm32f1xx_hal.h"
6  #include "diskio.h"
7  #include "fatfs_sd.h"
8
9  uint16_t Timer1, Timer2;           /* 1ms Timer Counter */
10
11 static volatile DSTATUS Stat = STA_NOINIT; /* Disk Status */
12 static uint8_t CardType;           /* Type 0:MMC, 1:SDC, 2:Block addressing */
13 static uint8_t PowerFlag = 0;      /* Power flag */
14
15 /*===== SD Functions =====*/
16
17 /* slave select */
18 static void SELECT(void)
19 {
20     HAL_GPIO_WritePin(SD_CS_PORT, SD_CS_PIN, GPIO_PIN_RESET);
21     HAL_Delay(1);
22 }
23
24 /* slave de-select */
25 static void DESELECT(void)
26 {
27     HAL_GPIO_WritePin(SD_CS_PORT, SD_CS_PIN, GPIO_PIN_SET);
28     HAL_Delay(1);
29 }
30
31 /* SPI transmit a byte */
32 static void SPI_TxByte(uint8_t data)
33 {
34     while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_TXE));
35     HAL_SPI_Transmit(HSPI_SDCARD, &data, 1, SPI_TIMEOUT);
36 }
37
38 /* SPI transmit buffer */
39 static void SPI_TxBuffer(uint8_t *buffer, uint16_t len)
40 {
41     while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_TXE));
42     HAL_SPI_Transmit(HSPI_SDCARD, buffer, len, SPI_TIMEOUT);
43 }
44
45 /* SPI receive a byte */
46 static uint8_t SPI_RxByte(void)
47 {
48     uint8_t dummy;
49     dummy = 0xFF;
50     while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_RXNE));
51     HAL_SPI_Receive(HSPI_SDCARD, &dummy, 1, SPI_TIMEOUT);
52     return dummy;
53 }
54
55 /* SPI receive a byte via pointer */
56 static void SPI_RxBytePtr(uint8_t *buff)
57 {
58     *buff = SPI_RxByte();
59 }
60
61 /*===== SD Functions =====*/
62
63 /* wait SD ready */
64 static uint8_t SD_ReadyWait(void)
65 {
66     uint8_t res;
67     /* timeout 500ms */
68     Timer2 = 500;
69     /* if SD goes ready, receives 0xFF */
70     do {
71         res = SPI_RxByte();
72     } while ((res != 0xFF) && Timer2);
73     return res;
74 }
75
76 /* power on */
77 static void SD_PowerOn(void)
78 {
79     /* ... */
80 }

```

```

81
82 /* SPI transmit buffer */
83 static void SPI_TxBuffer(uint8_t *buffer, uint16_t len)
84 {
85     while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_TXE));
86     HAL_SPI_Transmit(HSPI_SDCARD, buffer, len, SPI_TIMEOUT);
87 }
88
89 /* SPI receive a byte */
90 static uint8_t SPI_RxByte(void)
91 {
92     uint8_t dummy;
93     dummy = 0xFF;
94     while(!__HAL_SPI_GET_FLAG(HSPI_SDCARD, SPI_FLAG_RXNE));
95     HAL_SPI_Receive(HSPI_SDCARD, &dummy, 1, SPI_TIMEOUT);
96     return dummy;
97 }
98
99 /* SPI receive a byte via pointer */
100 static void SPI_RxBytePtr(uint8_t *buff)
101 {
102     *buff = SPI_RxByte();
103 }
104
105 /*===== SD Functions =====*/
106
107 /* wait SD ready */
108 static uint8_t SD_ReadyWait(void)
109 {
110     uint8_t res;
111     /* timeout 500ms */
112     Timer2 = 500;
113     /* if SD goes ready, receives 0xFF */
114     do {
115         res = SPI_RxByte();
116     } while ((res != 0xFF) && Timer2);
117     return res;
118 }
119
120 /* power on */
121 static void SD_PowerOn(void)
122 {
123     /* ... */
124 }

```

```

171
172
173 do {
174     res = SPI_RxByte();
175     } while ((res != 0xFF) && Timer2);
176
177     return res;
178 }
179
180 /* power on */
181 static void SD_PowerOn(void)
182 {
183     uint8_t args[6];
184     uint32_t cnt = 0xFFFF;
185
186     /* transmit bytes to wake up */
187     DESELECT();
188     for(int i = 0; i < 10; i++)
189     {
190         SPI_TxByte(0xFF);
191     }
192
193     /* slave select */
194     SELECT();
195
196     /* make idle state */
197     args[0] = CMD0; /* CMD0:GO_IDLE_STATE */
198     args[1] = 0;
199     args[2] = 0;
200     args[3] = 0;
201     args[4] = 0;
202     args[5] = 0x95; /* CRC */
203
204     SPI_TxBuffer(args, sizeof(args));
205
206     /* wait response */
207     while ((SPI_RxByte() != 0x01) && cnt)
208     {
209         cnt--;
210     }
211
212     DESELECT();
213     SPI_TxByte(0xFF);
214
215     PowerFlag = 1;
216
217     /* power off */
218     static void SD_PowerOff(void)
219     {
220         PowerFlag = 0;
221     }
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

167  uint8_t i = 0;
168
169  /* wait SD ready */
170  if (SD_ReadyWait() != 0xFF) return FALSE;
171
172  /* transmit token */
173  SPI_TxByte(token);
174
175  /* if it's not STOP token, transmit data */
176  if (token != 0xFF)
177  {
178      SPI_TxBuffer((uint8_t*)buff, 512);
179
180      /* discard CRC */
181      SPI_RxByte();
182      SPI_RxByte();
183
184      /* receive response */
185      while (i <= 64)
186      {
187          resp = SPI_RxByte();
188
189          /* transmit 0x05 accepted */
190          if ((resp & 0x1F) == 0x05) break;
191          i++;
192      }
193
194      /* zero buffer clear */
195      while (SPI_RxByte() == 0);
196  }
197
198  /* transmit 0x05 accepted */
199  if ((resp & 0x1F) == 0x05) return TRUE;
200
201  return FALSE;
202
203 #endif /* USE_WRITE */
204
205 /* Transmit command */
206 static BYTE SD_SendCmd(BYTE cmd, uint32_t arg)
207 {
208     uint8_t crc, res;
209
210     /* wait SD ready */
211     if (SD_ReadyWait() != 0xFF) return 0xFF;
212
213     /* transmit command */
214     SPI_TxByte(cmd);
215     SPI_TxByte((uint8_t)(arg >> 24)); /* Command */
216     SPI_TxByte((uint8_t)(arg >> 16)); /* Argument[31..24] */
217     SPI_TxByte((uint8_t)(arg >> 8)); /* Argument[23..16] */
218     SPI_TxByte((uint8_t)arg); /* Argument[15..0] */
219
220     /* prepare CRC */
221     if (cmd == CMD0) crc = 0x95; /* CRC for CMD0(0) */
222     else if (cmd == CMD8) crc = 0x87; /* CRC for CMD8(0x1AA) */
223     else crc = 1;
224
225     /* transmit CRC */
226     SPI_TxByte(crc);
227
228     /* skip a wait type when STOP_TRANSMISSION */
229     if (cmd == CMD12) SPI_RxByte();
230
231     /* receive response */
232     uint8_t n = 10;
233     do {
234         res = SPI_RxByte();
235     } while ((res & 0x80) && --n);
236
237     return res;
238 }
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

214  /* transmit command */
215  SPI_TxByte(cmd);
216  SPI_TxByte((uint8_t)(arg >> 24)); /* Command */
217  SPI_TxByte((uint8_t)(arg >> 16)); /* Argument[31..24] */
218  SPI_TxByte((uint8_t)(arg >> 8)); /* Argument[23..16] */
219  SPI_TxByte((uint8_t)arg); /* Argument[15..0] */
220
221  /* prepare CRC */
222  if (cmd == CMD0) crc = 0x95; /* CRC for CMD0(0) */
223  else if (cmd == CMD8) crc = 0x87; /* CRC for CMD8(0x1AA) */
224  else crc = 1;
225
226  /* transmit CRC */
227  SPI_TxByte(crc);
228
229  /* skip a wait type when STOP_TRANSMISSION */
230  if (cmd == CMD12) SPI_RxByte();
231
232  /* receive response */
233  uint8_t n = 10;
234  do {
235      res = SPI_RxByte();
236  } while ((res & 0x80) && --n);
237
238  return res;
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```



```

213  /* transmit command */
214  SPI_TxByte(cmd);
215  SPI_TxByte((uint8_t)(arg >> 24)); /* Command */
216  SPI_TxByte((uint8_t)(arg >> 16)); /* Argument[23..16] */
217  SPI_TxByte((uint8_t)(arg >> 8)); /* Argument[15..8] */
218  SPI_TxByte((uint8_t)arg); /* Argument[7..0] */
219
220  /* prepare CRC */
221  if(cmd == CMD0) crc = 0x95; /* CRC for CMD0(0) */
222  else if(cmd == CMD8) crc = 0x87; /* CRC for CMD8(0x1AA) */
223  else crc = 1;
224
225  /* transmit CRC */
226  SPI_TxByte(crc);
227
228  /* skip a stuff byte when STOP TRANSMISSION */
229  if (cmd == CMD12) SPI_RxByte();
230
231  /* receive response */
232  uint8_t n = 10;
233  do {
234      res = SPI_RxByte();
235      while ((res & 0x50) && --n);
236  } while (1);
237  return res;
238
239  =====
240  USER DISK I/O FUNCTIONS
241  =====
242
243  /* initialize SD */
244  STATUS SD_disk_initialize(BYTE drv)
245  {
246      uint8_t n, type, ocr[4];
247
248      /* single drive, drv should be 0 */
249      if(drv) return STA_NOINIT;
250
251      /* no disk */
252      if(Stat & STA_NODISK) return Stat;
253
254      /* power on */
255      SD_PowerOn();
256
257      /* slave select */
258      SELECT();
259
260      /* check disk type */
261      type = 0;
262
263      /* send SD_INIT_WRITE command */
264      if (SD_SendCmd(CMD0, 0) == 1)
265      {
266          /* timeout 1 sec */
267          Timer1 = 1000;
268
269          /* SDC V2+ accept CMD8 command, http://elm-chan.org/docs/mmc/mmc_w.html */
270          if (SD_SendCmd(CMD8, 0x1AA) == 1)
271          {
272              /* operation condition register */
273              for (n = 0; n < 4; n++)
274              {
275                  ocr[n] = SPI_RxByte();
276              }
277
278              /* Voltage range 2.7-3.6V */
279              if (ocr[0] == 0x01 && ocr[1] == 0xAA)
280              {
281                  /* ACMD41 with HCS bit */
282                  do {
283                      if (SD_SendCmd(CMD55, 0) <= 1 && SD_SendCmd(CMD41, 0) <= 1) break;
284                      while (Timer1);
285                  } while (1);
286
287                  /* READ OCR */
288                  if (Timer1 && SD_SendCmd(CMD58, 0) == 0)
289                  {
290                      /* Check OCR bit */
291                      for (n = 0; n < 4; n++)
292                      {
293                          ocr[n] = SPI_RxByte();
294                      }
295
296                      /* SDv2 (HC or SC) */
297                      type = (ocr[0] & 0x40) ? CT_SD2 : CT_BLOCK;
298                  }
299              }
300
301              /* SDv1 (MMC or SC) */
302              type = (SD_SendCmd(CMD55, 0) <= 1 && SD_SendCmd(CMD41, 0) <= 1) ? CT_SD1 : CT_MMC;
303
304              do
305

```

```

260      /* slave select */
261      SELECT();
262
263      /* check disk type */
264      type = 0;
265
266      /* send SD_INIT_WRITE command */
267      if (SD_SendCmd(CMD0, 0) == 1)
268      {
269          /* timeout 1 sec */
270          Timer1 = 1000;
271
272          /* SDC V2+ accept CMD8 command, http://elm-chan.org/docs/mmc/mmc_w.html */
273          if (SD_SendCmd(CMD8, 0x1AA) == 1)
274          {
275              /* operation condition register */
276              for (n = 0; n < 4; n++)
277              {
278                  ocr[n] = SPI_RxByte();
279              }
280
281              /* Voltage range 2.7-3.6V */
282              if (ocr[0] == 0x01 && ocr[1] == 0xAA)
283              {
284                  /* ACMD41 with HCS bit */
285                  do {
286                      if (SD_SendCmd(CMD55, 0) <= 1 && SD_SendCmd(CMD41, 0) <= 1) break;
287                      while (Timer1);
288                  } while (1);
289
290                  /* READ OCR */
291                  if (Timer1 && SD_SendCmd(CMD58, 0) == 0)
292                  {
293                      /* Check OCR bit */
294                      for (n = 0; n < 4; n++)
295                      {
296                          ocr[n] = SPI_RxByte();
297                      }
298
299                      /* SDv2 (HC or SC) */
300                      type = (ocr[0] & 0x40) ? CT_SD2 : CT_BLOCK;
301                  }
302              }
303
304              /* SDv1 (MMC or SC) */
305              type = (SD_SendCmd(CMD55, 0) <= 1 && SD_SendCmd(CMD41, 0) <= 1) ? CT_SD1 : CT_MMC;
306
307              do
308

```

The screenshot shows the Keil uVision IDE interface. On the left is the Project Explorer showing the project structure for 'F103ck_SD_CARD'. The main editor displays the source code for 'F103ck_SD_CARD.c'. The code includes headers for 'stm32f10x_gpio.h', 'stm32f10x_rtc.h', 'stm32f10x_usart.h', and 'stm32f10x_sd.h'. It defines a 'uint8_t' type and a 'uint16_t' type. The code implements several functions: 'SD_Init' for initialization, 'SD_Status' for checking the status, 'SD_Read' for reading data from the SD card, and 'SD_Write' for writing data to the SD card. The code uses the 'SD_SendCmd' function to send commands to the SD card. The code is written in C and uses the 'Keil' compiler.

```

329 {
330     /* Initialization failed */
331     SD_PowerOff();
332 }
333
334 return Stat;
335 }
336
337 /* return disk status */
338 STATUS SD_disk_status(BYTE drv)
339 {
340     if (drv) return STA_NOINIT;
341     return Stat;
342 }
343
344 /* read sector */
345 RESULT SD_disk_read(BYTE pdrv, BYTE* buff, DWORD sector, UINT count)
346 {
347     /* pdrv should be 0 */
348     if (pdrv != 0) return RES_PARERR;
349
350     /* no disk */
351     if (Stat & STA_NOINIT) return RES_NOTRDY;
352
353     /* convert to byte address */
354     if (!(CardType & CT_SD2)) sector *= 512;
355
356     SELECT();
357
358     if (count == 1)
359     {
360         /* READ SINGLE BLOCK */
361         if ((SD_SendCmd(CMD17, sector) == 0) && SD_RxDataBlock(buff, 512)) count = 0;
362     }
363     else
364     {
365         /* READ MULTIPLE BLOCK */
366         if (SD_SendCmd(CMD18, sector) == 0)
367         {
368             do {
369                 if (!SD_RxDataBlock(buff, 512)) break;
370                 buff += 512;
371             } while (--count);
372         }
373     }
374
375     /* STOP TRANSMISSION */
376     SD_SendCmd(CMD12, 0);
377 }
378
379 }
380
381 }
  
```



```

386         } while (--count);
387     }
388     /* STOP_TRANSMISSION */
389     SD_SendCmd(CMD12, 0);
390 }
391
392 /* Idle */
393 DESELECT();
394 SPI_RxByte();
395
396 return count ? RES_ERROR : RES_OK;
397
398 /* write sector */
399 #if _USE_WRITE == 1
400 #define SD_disk_write(BYTE drv, const BYTE* buff, DWORD sector, UINT count)
401
402 /* drv should be 0 */
403 if (drv != 0) return RES_PARERR;
404
405 /* no disk */
406 if (Stat & STA_NOINIT) return RES_NOTRDY;
407
408 /* write protection */
409 if (Stat & STA_PROTECT) return RES_WRPRT;
410
411 /* convert to byte address */
412 if ((CardType & CT_SD2) && sector != 512)
413     SELECT();
414
415 if (count == 1)
416 {
417     /* WRITE BLOCK */
418     if ((SD_SendCmd(CMD24, sector) == 0) && SD_TxDataBlock(buff, 0xFF))
419         count = 0;
420 }
421 else
422 {
423     /* WRITE MULTIPLE BLOCK */
424     if (CardType & CT_SD1)
425     {
426         SD_SendCmd(CMD55, 0);
427         SD_SendCmd(CMD23, count); /* ACMD23 */
428     }
429
430     if (SD_SendCmd(CMD25, sector) == 0)
431     {
432

```

```

433     {
434         /* WRITE MULTIPLE BLOCK */
435         if (CardType & CT_SD1)
436         {
437             SD_SendCmd(CMD55, 0);
438             SD_SendCmd(CMD23, count); /* ACMD23 */
439         }
440
441         if (SD_SendCmd(CMD25, sector) == 0)
442         {
443             do {
444                 if (SD_TxDataBlock(buff, 0xFF)) break;
445                 buff += 512;
446             } while (--count);
447
448             /* STOP_TRAN token */
449             if (SD_TxDataBlock(0, 0xFF))
450             {
451                 count = 1;
452             }
453         }
454     }
455
456     /* Idle */
457     DESELECT();
458     SPI_RxByte();
459
460     return count ? RES_ERROR : RES_OK;
461 }
462
463 #endif /* _USE_WRITE */
464
465 /* ioctl */
466 #define SD_disk_ioctl(BYTE drv, BYTE ctrl, void *buff)
467
468 #define RES res;
469 uint8_t n, cmd[16], *ptr = buff;
470 WORD csize;
471
472 /* drv should be 0 */
473 if (drv) return RES_PARERR;
474 res = RES_ERROR;
475
476 if (ctrl == CTRL_POWER)
477 {
478     switch (*ptr)
479     {
480     case 0:
481         SD_PowerOff(); /* Power OFF */
482         res = RES_OK;
483     }
484 }

```

```

458 if (drv) return RES_PARERR;
459 res = RES_ERROR;
460
461 if (ctrl == CTRL_POWER)
462 {
463     switch (*ptr)
464     {
465         case 0:
466             SD_PowerOff(); /* Power OFF */
467             res = RES_OK;
468             break;
469         case 1:
470             SD_PowerOn(); /* Power ON */
471             res = RES_OK;
472             break;
473         case 2:
474             *ptr++ = SD_CheckPower();
475             res = RES_OK; /* Power Check */
476             break;
477         default:
478             res = RES_PARERR;
479     }
480 }
481 else
482 {
483     /* no disk */
484     if (Stat & STA_NOINIT) return RES_NOTRDY;
485
486     SELECT();
487
488     switch (ctrl)
489     {
490         case GET_SECTOR_COUNT:
491             /* SEND CMD */
492             if ((SD_SendCmd(CMD9, 0) == 0) && SD_RxDataBlock(ptr, 16))
493             {
494                 if ((caddr[0] >> 6) == 1)
495                 {
496                     /* SDC V2 */
497                     csize = caddr[9] + ((WORD) caddr[1] << 8) + 1;
498                     *(DWORD*) buff = (DWORD) csize << 10;
499                 }
500                 else
501                 {
502                     /* MMC or SDC V1 */
503                     n = (caddr[5] & 15) + ((caddr[10] & 128) >> 7) + ((caddr[9] & 3) << 1) + 2;
504                     csize = (caddr[5] >> 6) + ((WORD) caddr[7] << 2) + ((WORD) (caddr[6] & 3) << 10) + 1;
505                     *(DWORD*) buff = (DWORD) csize << (n - 9);
506                 }
507             }
508             else
509             {
510                 res = RES_ERROR;
511             }
512             break;
513         case GET_SECTOR_SIZE:
514             *(DWORD*) buff = 512;
515             res = RES_OK;
516             break;
517         case CTRL_SYNC:
518             if (SD_ReadyWait() == 0xFF) res = RES_ERROR;
519             break;
520         case MMC_GET_CSD:
521             /* SEND CMD */
522             if ((SD_SendCmd(CMD9, 0) == 0) && SD_RxDataBlock(ptr, 16)) res = RES_OK;
523             break;
524         case MMC_GET_CID:
525             /* SEND CMD */
526             if ((SD_SendCmd(CMD10, 0) == 0) && SD_RxDataBlock(ptr, 16)) res = RES_OK;
527             break;
528         case MMC_GET_OCR:
529             /* SEND CMD */
530             if ((SD_SendCmd(CMD58, 0) == 0)
531                 {
532                     for (n = 0; n < 4; n++)
533                     {
534                         *ptr++ = SPI_RxByte();
535                     }
536                     res = RES_OK;
537                 }
538             else
539             {
540                 res = RES_ERROR;
541             }
542             DESELECT();
543             SPI_RxByte();
544             return res;
545         }
546     }

```

```

547         csize = caddr[9] + ((WORD) caddr[1] << 8) + 1;
548         *(DWORD*) buff = (DWORD) csize << 10;
549     }
550     else
551     {
552         /* MMC or SDC V1 */
553         n = (caddr[5] & 15) + ((caddr[10] & 128) >> 7) + ((caddr[9] & 3) << 1) + 2;
554         csize = (caddr[5] >> 6) + ((WORD) caddr[7] << 2) + ((WORD) (caddr[6] & 3) << 10) + 1;
555         *(DWORD*) buff = (DWORD) csize << (n - 9);
556     }
557     res = RES_OK;
558 }
559 break;
560 case GET_SECTOR_SIZE:
561     *(DWORD*) buff = 512;
562     res = RES_OK;
563     break;
564 case CTRL_SYNC:
565     if (SD_ReadyWait() == 0xFF) res = RES_ERROR;
566     break;
567 case MMC_GET_CSD:
568     /* SEND CMD */
569     if ((SD_SendCmd(CMD9, 0) == 0) && SD_RxDataBlock(ptr, 16)) res = RES_OK;
570     break;
571 case MMC_GET_CID:
572     /* SEND CMD */
573     if ((SD_SendCmd(CMD10, 0) == 0) && SD_RxDataBlock(ptr, 16)) res = RES_OK;
574     break;
575 case MMC_GET_OCR:
576     /* SEND CMD */
577     if ((SD_SendCmd(CMD58, 0) == 0)
578         {
579             for (n = 0; n < 4; n++)
580             {
581                 *ptr++ = SPI_RxByte();
582             }
583             res = RES_OK;
584         }
585     else
586     {
587         res = RES_ERROR;
588     }
589     DESELECT();
590     SPI_RxByte();
591     return res;
592 }

```

Figure 4.2: Library Fatfs_sd.c

4.3. STM32f1xx_it.c

The previously mentioned 10ms timers Timer1 and Timer2 are handled by SysTick_Handler as shown below. File code here [stm32f1xx_it.c](#) .

```

1  Private variables -----
2  USER CODE BEGIN PV **
3
4  USER CODE END PV **
5
6  Private function prototypes -----
7  USER CODE BEGIN PFP **
8
9  USER CODE END PFP **
10
11 Private user code -----
12 USER CODE BEGIN 0 **
13
14 extern uint16_t Timer1, Timer2;
15
16 USER CODE END 0 **
17
18 External variables -----
19 USER CODE BEGIN EV **
20
21 USER CODE END EV **
22
23 =====
24 Cortex-M3 Processor Interruption and Exception Handlers
25 =====
26
27 * Brief This function handles Non maskable interrupt.
28
29 void NMI_Handler(void)
30 {
31     /* USER CODE BEGIN NonMaskableInt_IRQn 0 */
32
33     /* USER CODE END NonMaskableInt_IRQn 0 */
34
35     while (1)
36     {
37     }
38 }
39
40 * Brief This function handles Hard fault interrupt.
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

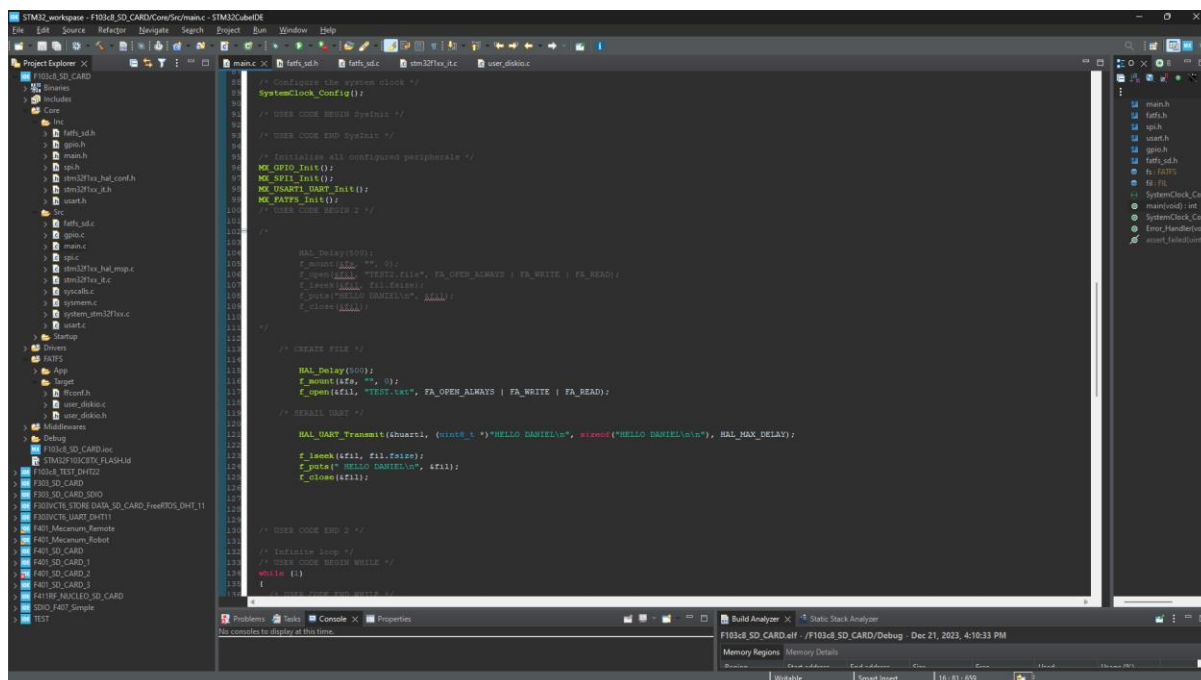
```

163
164 /* USER CODE BEGIN DebugMonitor_IRQn 0 */
165
166 /* USER CODE END DebugMonitor_IRQn 0 */
167
168
169 * Brief This function handles Pendable request for system service.
170
171 void PendSV_Handler(void)
172 {
173     /* USER CODE BEGIN PendSV_IRQn 0 */
174
175     /* USER CODE END PendSV_IRQn 0 */
176
177     while (1)
178     {
179     }
180 }
181
182 * Brief This function handles System tick timer.
183
184 void SysTick_Handler(void)
185 {
186     /* USER CODE BEGIN SysTick_IRQn 0 */
187
188     if (Timer1 > 0)
189     {
190         Timer1--;
191     }
192     if (Timer2 > 0)
193     {
194         Timer2--;
195     }
196     /* USER CODE END SysTick_IRQn 0 */
197     HAL_IncTick();
198     /* USER CODE BEGIN SysTick_IRQn 1 */
199
200     /* USER CODE END SysTick_IRQn 1 */
201 }
202
203 =====
204 STM32F1xx Peripheral Interrupt Handlers
205 =====
206
207 * Add here the Interrupt Handlers for the used peripherals.
208 * For the available peripheral interrupt handler names,
209 * please refer to the startup file (startup_stm32f1xx.s).
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Figure 4.3: STM32f1xx_it.c

Now that all driver-related work has been completed, let's write code that simply creates a text file using the FatFS API function, writes the contents, and reads the file again. File code example here [main.c](#)



18

5. Result Testing

- USB-Serial UART

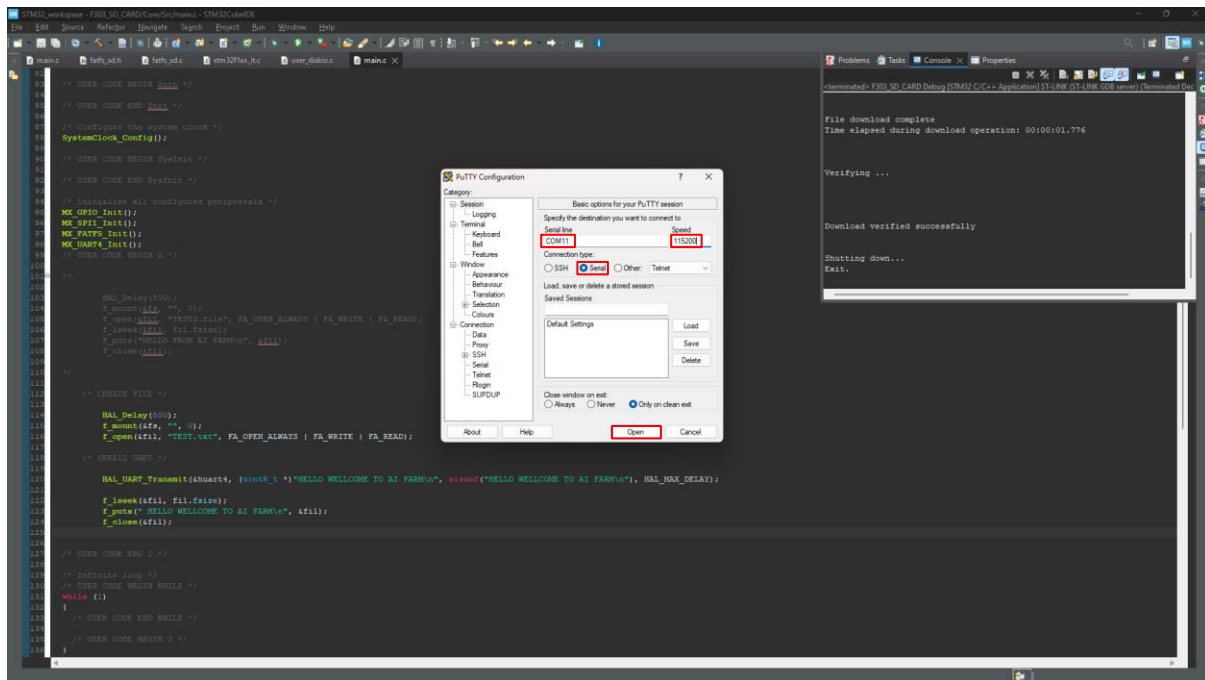


Figure 5.1: USB-Serial UART

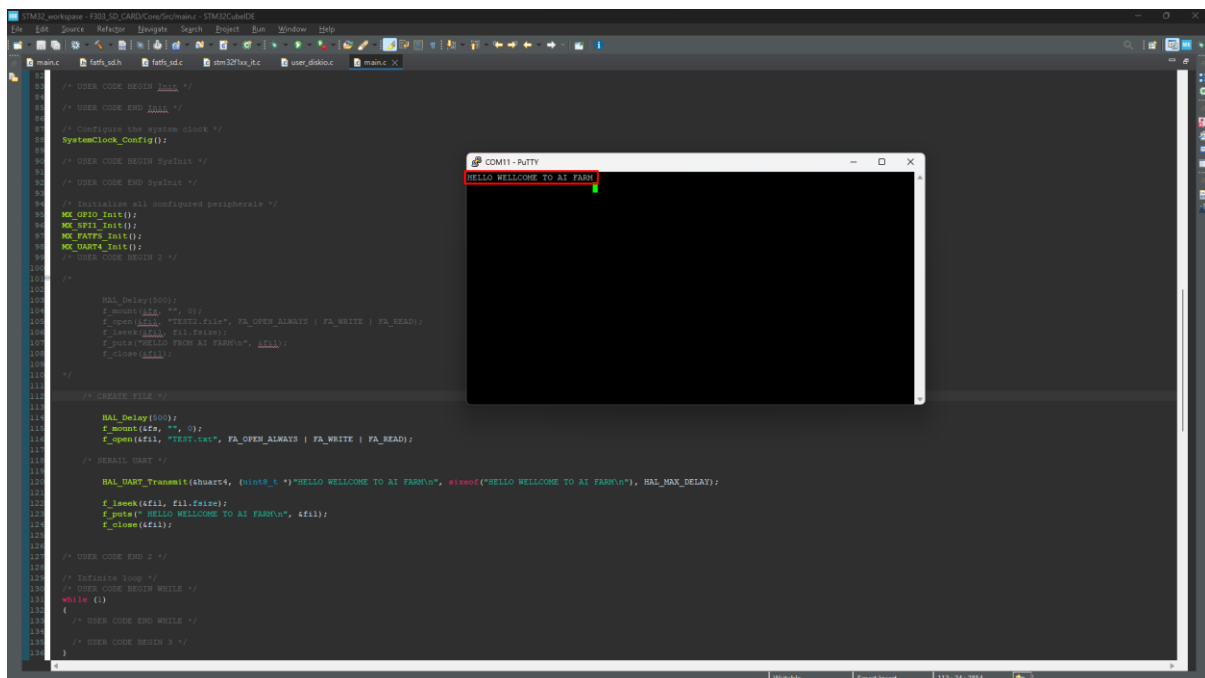


Figure 5.2: USB-Serial UART PUTTY

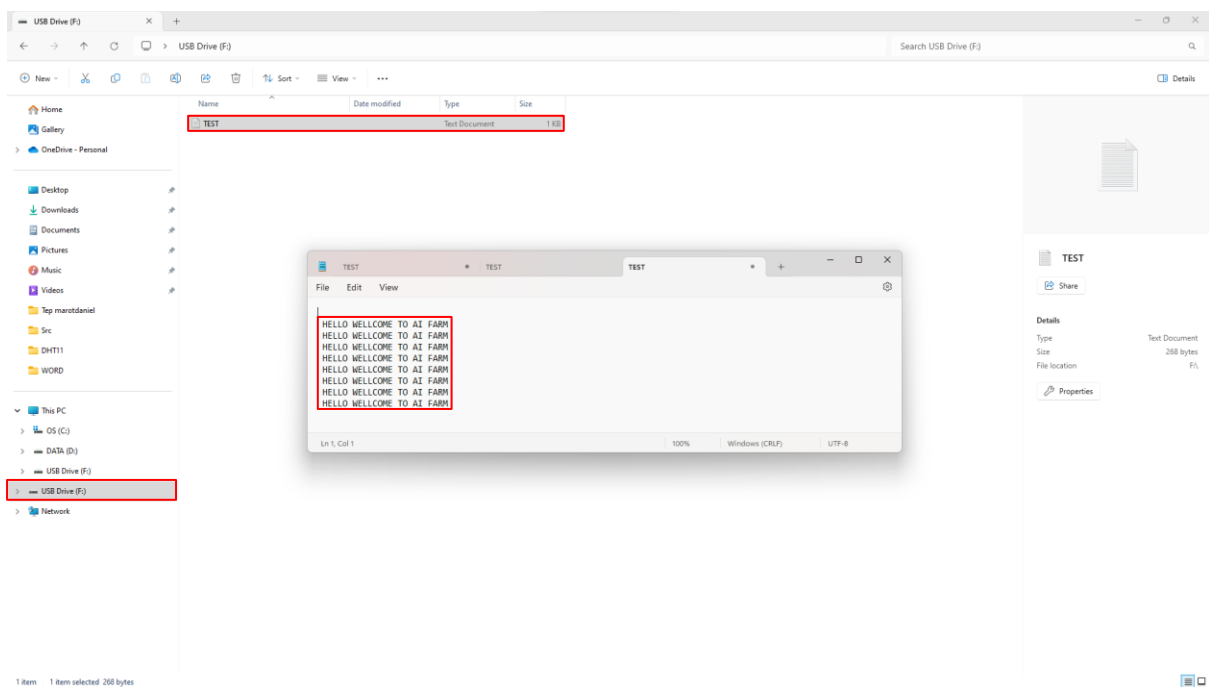


Figure 5.3: USB Drive (F) from SD CARD

6. Resources

- [1] [Eziya's](#)
- [2] [Controllerstech](#)
- [3] [Kiwih](#)
- [4] [EmbeTronicX](#)
- [5] [MicroPeta](#)

APPENDIX

Link Project here [STM32 SD CARD](#)